

Chapter 12

Analysis of Mindlin plates

12.1 Introduction

This chapter considers the static analysis of Mindlin plates in bending. Also, we compute the free vibration problem and the buckling problem.

12.2 The Mindlin plate theory

The Mindlin plate theory or first-order shear deformation theory for plates includes the effect of transverse shear deformations [18]. It may be considered an extension of the Timoshenko theory for beams in bending. The main difference for thin, Kirchhoff-type theories is that in the Mindlin theory the normals to the undeformed middle plane of the plate remain straight, but not normal to the deformed middle surface.

The strain energy of the Mindlin plate is given as [15, 18]

$$U = \frac{1}{2} \int_V \boldsymbol{\sigma}_f^T \boldsymbol{\epsilon}_f dV + \frac{\alpha}{2} \int_V \boldsymbol{\sigma}_c^T \boldsymbol{\epsilon}_c dV \quad (12.1)$$

where

$$\boldsymbol{\sigma}_f^T = [\sigma_x \ \sigma_y \ \tau_{xy}] \quad (12.2)$$

$$\boldsymbol{\epsilon}_f^T = [\epsilon_x \ \epsilon_y \ \gamma_{xy}] \quad (12.3)$$

are the bending stresses and strains, and

$$\boldsymbol{\sigma}_c^T = [\tau_{xz} \ \tau_{yz}] \quad (12.4)$$

$$\boldsymbol{\epsilon}_c^T = [\gamma_{xz} \ \gamma_{yz}] \quad (12.5)$$

are the transverse shear stresses and strains. The α parameter, also known as the shear correction factor can be taken as 5/6 [4].

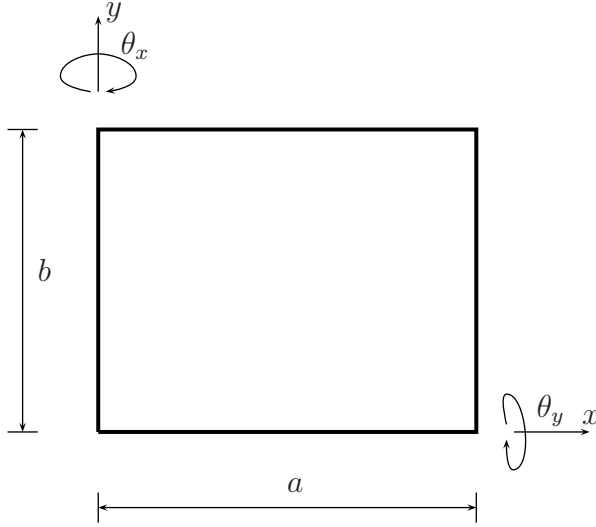


Fig. 12.1 Mindlin plate: illustration of geometry, degrees of freedom

The assumed displacement field for a thick plate (thickness h) is defined as

$$u = z\theta_x; \quad v = z\theta_y; \quad w = w_0 \quad (12.6)$$

where θ_x, θ_y are the rotations of the normal to the middle plane with respect to axes y and x , respectively (as illustrated in figure 12.1).

12.2.1 Strains

Bending strains are obtained as

$$\epsilon_x = \frac{\partial u}{\partial x} = z \frac{\partial \theta_x}{\partial x} \quad (12.7)$$

$$\epsilon_y = \frac{\partial v}{\partial y} = z \frac{\partial \theta_y}{\partial y} \quad (12.8)$$

$$\gamma_{xy} = \frac{\partial u}{\partial y} + \frac{\partial v}{\partial x} = z \left(\frac{\partial \theta_y}{\partial x} + \frac{\partial \theta_x}{\partial y} \right) \quad (12.9)$$

while the transverse shear deformations are obtained as

$$\gamma_{xz} = \frac{\partial w}{\partial x} + \frac{\partial u}{\partial z} = \frac{\partial w}{\partial x} + \theta_x \quad (12.10)$$

$$\gamma_{yz} = \frac{\partial w}{\partial y} + \frac{\partial v}{\partial z} = \frac{\partial w}{\partial y} + \theta_y \quad (12.11)$$

12.2.2 Stresses

The linear elastic stress-strain relations in bending are defined for a homogeneous, isotropic material as

$$\boldsymbol{\sigma}_f = \mathbf{D}_f \boldsymbol{\epsilon}_f \quad (12.12)$$

where $\mathbf{D}_f$ is defined as

$$\mathbf{D}_f = \frac{E}{1 - \nu^2} \begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & \frac{1 - \nu}{2} \end{bmatrix} \quad (12.13)$$

while the linear elastic stress-strain relations in transverse shear are defined as

$$\boldsymbol{\sigma}_c = \mathbf{D}_c \boldsymbol{\epsilon}_c \quad (12.14)$$

where

$$\mathbf{D}_c = \begin{bmatrix} G & 0 \\ 0 & G \end{bmatrix} \quad (12.15)$$

where G the shear modulus. Introducing these concepts into the strain energy (12.1), we obtain

$$U = \frac{1}{2} \int_V \boldsymbol{\epsilon}_f^T \mathbf{D}_f \boldsymbol{\epsilon}_f dV + \frac{\alpha}{2} \int_V \boldsymbol{\epsilon}_c^T \mathbf{D}_c \boldsymbol{\epsilon}_c dV \quad (12.16)$$

12.3 Finite element discretization

The generalized displacements are independently interpolated using the same shape functions

$$w = \sum_{i=1}^n N_i(\xi, \eta) w_i, \quad \theta_x = \sum_{i=1}^n N_i(\xi, \eta) \theta_{xi}, \quad \theta_y = \sum_{i=1}^n N_i(\xi, \eta) \theta_{yi} \quad (12.17)$$

where $N_i(\xi, \eta)$ are the shape functions of a bilinear four-noded Q4 element.

Strains are defined as

$$\boldsymbol{\epsilon}_f = z \mathbf{B}_f \mathbf{d}^e; \quad \boldsymbol{\epsilon}_c = \mathbf{B}_c \mathbf{d}^e \quad (12.18)$$

The strain-displacement matrices for bending and shear contributions are obtained by derivation of the shape functions by

$$\mathbf{B}_f = \begin{bmatrix} 0 & \frac{\partial N_1}{\partial x} & 0 & \dots & 0 & \frac{\partial N_4}{\partial x} & 0 \\ 0 & 0 & \frac{\partial N_1}{\partial y} & \dots & 0 & 0 & \frac{\partial N_4}{\partial y} \\ 0 & \frac{\partial N_1}{\partial y} & \frac{\partial N_1}{\partial x} & \dots & 0 & \frac{\partial N_4}{\partial y} & \frac{\partial N_4}{\partial x} \end{bmatrix} \quad (12.19)$$

$$\mathbf{B}_c = \begin{bmatrix} \frac{\partial N_1}{\partial x} & N_1 & 0 & \dots & \frac{\partial N_4}{\partial x} & N_4 & 0 \\ \frac{\partial N_1}{\partial y} & 0 & N_1 & \dots & \frac{\partial N_4}{\partial y} & 0 & N_4 \end{bmatrix} \quad (12.20)$$

where

$$\mathbf{d}^e = \{w_1 \quad \theta_{x1} \quad \theta_{y1} \dots \quad w_4 \quad \theta_{x4} \quad \theta_{y4}\} \quad (12.21)$$

We then obtain the plate strain energy as

$$U = \frac{1}{2} \mathbf{d}^e T \int_{\Omega^e} \int_z \mathbf{B}_f^T \mathbf{D}_f \mathbf{B}_f dz d\Omega^e \mathbf{d}^e + \frac{\alpha}{2} \mathbf{d}^e T \int_{\Omega^e} \int_z \mathbf{B}_c^T \mathbf{D}_c \mathbf{B}_c dz d\Omega^e \mathbf{d}^e \quad (12.22)$$

The stiffness matrix of the Mindlin plate is then obtained as

$$\mathbf{K}^e = \frac{h^3}{12} \int_{\Omega^e} \mathbf{B}_f^T \mathbf{D}_f \mathbf{B}_f d\Omega^e + \alpha h \int_{\Omega^e} \mathbf{B}_c^T \mathbf{D}_c \mathbf{B}_c d\Omega^e \quad (12.23)$$

or

$$\mathbf{K}^e = \frac{h^3}{12} \int_{-1}^1 \int_{-1}^1 \mathbf{B}_f^T D_f \mathbf{B}_f |J| d\xi d\eta + \alpha h \int_{-1}^1 \int_{-1}^1 \mathbf{B}_c^T D_c \mathbf{B}_c |J| d\xi d\eta \quad (12.24)$$

where $|J|$ is the determinant of the Jacobian matrix.

The vector of nodal forces equivalent to distributed forces P is defined as

$$\mathbf{f}^e = \int_{-1}^1 \int_{-1}^1 \mathbf{N} P |J| d\xi d\eta \quad (12.25)$$

Both the stiffness matrix and the force vector integrals are computed by numerical integration. The stiffness integral is solved by considering for the Q4 element, 2×2 Gauss points for the bending contribution and 1 point for the shear contribution. This selective integration proved to be one of the simplest remedies for avoiding shear locking [4, 12].

12.4 Example: a square Mindlin plate in bending

We consider a simply-supported and clamped square plate (side $a = 1$) under uniform transverse pressure ($P = 1$), and thickness h . The modulus of elasticity is taken $E = 10,920^1$ and the Poisson's ratio is taken as $\nu = 0.3$. The non-dimensional transverse displacement is set as

$$\bar{w} = w \frac{D}{Pl^4} \quad (12.26)$$

where the bending stiffness D is taken as

$$D = \frac{Eh^3}{12(1 - \nu^2)} \quad (12.27)$$

In table 12.1 we present non-dimensional transverse displacement results obtained by the code `problem19.m` for various thickness values and boundary conditions. In figure 12.2 we show the deformed shape of a simply-supported plate, using a 20×20 Q4 mesh.

```
%.....

% MATLAB codes for Finite Element Analysis
% problem19.m
```

Table 12.1 Non-dimensional transverse displacement of a square plate, under uniform pressure – simply-supported (SSSS) and clamped (CCCC) boundary conditions

a/h	Mesh	SSSS	CCCC
10	2×2	0.003545	0.000357
	6×6	0.004245	0.001486
	10×10	0.004263	0.001498
	20×20	0.004270	0.001503
	30×30	0.004271	0.001503
	Exact solution	0.004270	
10,000	2×2	0.003188	$3.5e^{-10}$
	6×6	0.004024	0.001239
	10×10	0.004049	0.001255
	20×20	0.004059	0.001262
	30×30	0.004060	0.001264
	Exact solution	0.004060	0.001260

¹ The reader may be curious about the reason for this particular value of E . With $a = 1$, thickness $h = 0.1$ and the mentioned values for E and ν we obtain a flexural stiffness of 1. This is only a practical convenience for non-dimensional results, not really a meaningful value.

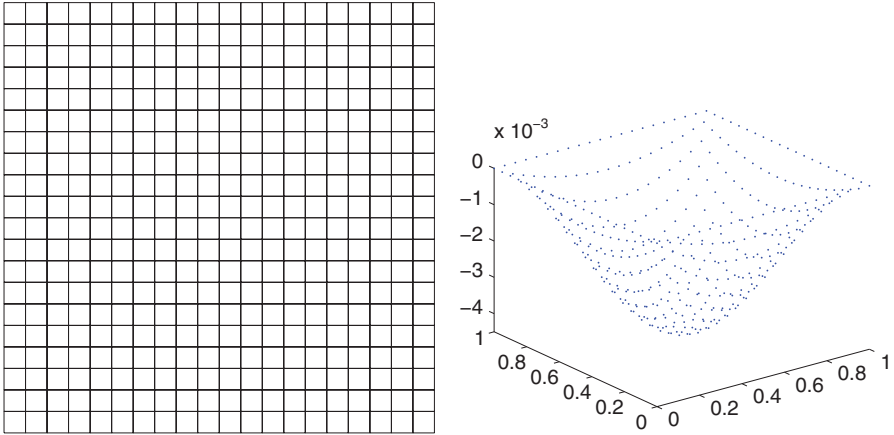


Fig. 12.2 Mesh of 20×20 Q4 elements and deformed shape

```
% Mindlin plate in bending
% antonio ferreira 2008

% clear memory
clear all; colordef white; clf

% materials
E = 10920;      poisson = 0.30; kapa=5/6;
thickness=0.1;
I=thickness^3/12;

% matrix C
% bending part
C_bending=I*E/(1-poisson^2)*...
    [1 poisson 0;poisson 1 0;0 0 (1-poisson)/2];
% shear part
C_shear=kapa*thickness*E/2/(1+poisson)*eye(2);

% load
P = -1;

%Mesh generation
L = 1;
numberElementsX=20;
numberElementsY=20;
numberElements=numberElementsX*numberElementsY;
```

```

%
[nodeCoordinates, elementNodes] = ...
    rectangularMesh(L,L,numberElementsX,numberElementsY);
xx=nodeCoordinates(:,1);
yy=nodeCoordinates(:,2);
drawingMesh(nodeCoordinates,elementNodes,'Q4','k-');
axis off
numberNodes=size(xx,1);

% GDof: global number of degrees of freedom
GDof=3*numberNodes;

% computation of the system stiffness matrix and force vector
[stiffness]=...
    formStiffnessMatrixMindlinQ4(GDof,numberElements,...
    elementNodes,numberNodes,nodeCoordinates,C_shear,...
    C_bending,thickness,I);

[force]=...
    formForceVectorMindlinQ4(GDof,numberElements,...
    elementNodes,numberNodes,nodeCoordinates,P);

% % boundary conditions
[prescribedDof,activeDof]=...
    EssentialBC('ssss',GDof,xx,yy,nodeCoordinates,numberNodes);

% solution
displacements=solution(GDof,prescribedDof,stiffness,force);

% displacements
disp('Displacements')
jj=1:GDof; format
f=[jj; displacements'];
fprintf('node U\n')
fprintf('%3d %12.8f\n',f)

% deformed shape
figure
plot3(xx,yy,displacements(1:numberNodes),'.')
format long
D1=E*thickness^3/12/(1-poisson^2);
min(displacements(1:numberNodes))*D1/L^4

```

This MATLAB code calls functions `formStiffnessMatrixMindlinQ4.m` for computation of stiffness matrix and `formForceVectorMindlinQ4.m` for computation of the force vector.

```
%.....

function [K]=...
    formStiffnessMatrixMindlinQ4(GDof,numberElements,...
    elementNodes,numberNodes,nodeCoordinates,C_shear,...
    C_bending,thickness,I)

% computation of stiffness matrix
% for Mindlin plate element

% K : stiffness matrix

K=zeros(GDof);

% Gauss quadrature for bending part
[gaussWeights,gaussLocations]=gaussQuadrature('complete');

% cycle for element
% cycle for element
for e=1:numberElements
    % indice : nodal condofectivities for each element
    % elementDof: element degrees of freedom
    indice=elementNodes(e,:);
    elementDof=[indice indice+numberNodes indice+2*numberNodes];
    ndof=length(indice);

    % cycle for Gauss point
    for q=1:size(gaussWeights,1)
        GaussPoint=gaussLocations(q,:);
        xi=GaussPoint(1);
        eta=GaussPoint(2);

% shape functions and derivatives
        [shapeFunction,naturalDerivatives]=shapeFunctionQ4(xi,eta)

% Jacobian matrix, inverse of Jacobian,
% derivatives w.r.t. x,y
        [Jacob,invJacobian,XYderivatives]=...
            Jacobian(nodeCoordinates(indice,:),naturalDerivatives);
```



```

% [B] matrix bending
B_b=zeros(3,3*ndof);
B_b(1,ndof+1:2*ndof) = XYderivatives(:,1)';
B_b(2,2*ndof+1:3*ndof)= XYderivatives(:,2)';
B_b(3,ndof+1:2*ndof) = XYderivatives(:,2)';
B_b(3,2*ndof+1:3*ndof)= XYderivatives(:,1)';

% stiffness matrix bending
K(elementDof,elementDof)=K(elementDof,elementDof)+ ...
    B_b'*C_bending*B_b*gaussWeights(q)*det(Jacob);
end % Gauss point
end % element

% shear stiffness matrix

% Gauss quadrature for shear part
[gaussWeights,gaussLocations]=gaussQuadrature('reduced');

% cycle for element
% cycle for element
for e=1:numberElements
    % indice : nodal condofectivities for each element
    % elementDof: element degrees of freedom
    indice=elementNodes(e,:);
    elementDof=[indice indice+numberNodes indice+2*numberNodes];
    ndof=length(indice);

    % cycle for Gauss point
    for q=1:size(gaussWeights,1)
        GaussPoint=gaussLocations(q,:);
        xi=GaussPoint(1);
        eta=GaussPoint(2);

% shape functions and derivatives
        [shapeFunction,naturalDerivatives]=shapeFunctionQ4(xi,eta)

% Jacobian matrix, inverse of Jacobian,
% derivatives w.r.t. x,y
        [Jacob,invJacobian,XYderivatives]=...
            Jacobian(nodeCoordinates(indice,:),naturalDerivatives);
% [B] matrix shear
        B_s=zeros(2,3*ndof);

```

```

    B_s(1,1:ndof)      = XYderivatives(:,1)';
    B_s(2,1:ndof)      = XYderivatives(:,2)';
    B_s(1,ndof+1:2*ndof) = shapeFunction;
    B_s(2,2*ndof+1:3*ndof)= shapeFunction;

% stiffness matrix shear
    K(elementDof,elementDof)=K(elementDof,elementDof)+...
        B_s'*C_shear *B_s*gaussWeights(q)*det(Jacob);
end % gauss point
end % element

%.....

function [force]=...
    formForceVectorMindlinQ4(GDof,numberElements,...
        elementNodes,numberNodes,nodeCoordinates,P)

% computation of force vector
% for Mindlin plate element

% force : force vector
force=zeros(GDof,1);

% Gauss quadrature for bending part
[gaussWeights,gaussLocations]=gaussQuadrature('reduced');

% cycle for element
for e=1:numberElements
    % indice : nodal connectivities for each element
    indice=elementNodes(e,:);

    % cycle for Gauss point
    for q=1:size(gaussWeights,1)
        GaussPoint=gaussLocations(q,:);
        GaussWeight=gaussWeights(q);
        xi=GaussPoint(1);
        eta=GaussPoint(2);

% shape functions and derivatives
        [shapeFunction,naturalDerivatives]=shapeFunctionQ4(xi,eta)
    end
end

```

```

% Jacobian matrix, inverse of Jacobian,
% derivatives w.r.t. x,y
    [Jacob,invJacobian,XYderivatives]=...
        Jacobian(nodeCoordinates(indice,:),naturalDerivatives);

% force vector
force(indice)=force(indice)+shapeFunction*P*det(Jacob)
    *GaussWeight;
end % Gauss point

end % element

```

The imposition of the essential boundary conditions is made in function `EssentialBC.m`, as:

```

%.....

function [prescribedDof,activeDof,fixedNodeW]=...
    EssentialBC(typeBC,GDof,xx,yy,nodeCoordinates,numberNodes)

% essentialBoundary conditions for rectangular plates
switch typeBC
    case 'ssss'
        fixedNodeW =find(xx==max(nodeCoordinates(:,1))|...
            xx==min(nodeCoordinates(:,1))|...
            yy==min(nodeCoordinates(:,2))|...
            yy==max(nodeCoordinates(:,2)));

        fixedNodeTX =find(yy==max(nodeCoordinates(:,2))|...
            yy==min(nodeCoordinates(:,2)));
        fixedNodeTY =find(xx==max(nodeCoordinates(:,1))|...
            xx==min(nodeCoordinates(:,1)));
        case 'cccc'

        fixedNodeW =find(xx==max(nodeCoordinates(:,1))|...
            xx==min(nodeCoordinates(:,1))|...
            yy==min(nodeCoordinates(:,2))|...
            yy==max(nodeCoordinates(:,2)));
        fixedNodeTX =fixedNodeW;
        fixedNodeTY =fixedNodeTX;
        case 'scsc'

```

```

fixedNodeW =find(xx==max(nodeCoordinates(:,1))|...
                xx==min(nodeCoordinates(:,1))|...
                yy==min(nodeCoordinates(:,2))|...
                yy==max(nodeCoordinates(:,2)));

fixedNodeTX =find(xx==max(nodeCoordinates(:,2))|...
                xx==min(nodeCoordinates(:,2)));
fixedNodeTY=[];
    case 'cccf'
fixedNodeW =find(xx==min(nodeCoordinates(:,1))|...
                yy==min(nodeCoordinates(:,2))|...
                yy==max(nodeCoordinates(:,2)));

fixedNodeTX =fixedNodeW;
fixedNodeTY =fixedNodeTX;
end

prescribedDof=[fixedNodeW;fixedNodeTX+numberNodes;...
               fixedNodeTY+2*numberNodes]
activeDof=setdiff([1:GDof]',[prescribedDof]);

```

The Mindlin plate `problem19` can be solved using MATLAB structures. Code `problem19structure.m` includes all the functions needed for the execution of the problem.

```

%.....

% MATLAB codes for Finite Element Analysis
% problem19structure.m
% Mindlin plate in bending
% antonio ferreira 2008
function problem19structure
% clear memory
clear all; colordef white; clf

% materials
E = 10920;      poisson = 0.30; kapa=5/6;
thickness=0.001;
I=thickness^3/12;

% load

```

```

element=struct('P',-1);

%Mesh generation
L = 1;
numberElementsX=20; numberElementsY=20;
element.numberElements=numberElementsX*numberElementsY;
[element.nodeCoordinates, element.elementNodes] = ...
    rectangularMesh(L,L,numberElementsX,numberElementsY);
element.numberNodes=size(element.nodeCoordinates,1);
element.GDof=3*size(element.nodeCoordinates,1);

% matrix C
% bending part :C_bending
% shear part : C_shear
element.C_bending=...
    I*E/(1-poisson^2)*[1 poisson 0;poisson 1 0;0 0 (1-poisson)/2];
element.C_shear=...
    kapa*thickness*E/2/(1+poisson)*eye(2);

% computation of the system stiffness matrix and force vector
element.stiffness=formStiffnessMatrixMindlinQ4Structure(element);
element.force=formForceVectorMindlinQ4Structure(element);

% % boundary conditions
element.prescribed=EssentialBCStructure('ssss',element);

% solution
element.displacements=solutionStructure(element);

% displacements
disp('Displacements')
jj=1:element.GDof;
f=[jj; element.displacements'];
fprintf('node U\n')
fprintf('%3d %12.8f\n',f)

% original mesh
drawingMesh(element.nodeCoordinates,element.elementNodes,'Q4',
    'k-');
axis off

% deformed shape
figure

```

```

plot3(element.nodeCoordinates(:,1),element.nodeCoordinates(:,2)...
      ,element.displacements(1:size(element.nodeCoordinates,1)),'.')
format long
D1=E*thickness^3/12/(1-poisson^2);
min(element.displacements(1:size(element.nodeCoordinates,1)))
      *D1/L^4

end

%.....
function [K]=formStiffnessMatrixMindlinQ4Structure(element)

% computation of stiffness matrix
% for Mindlin plate element

% K : stiffness matrix
K=zeros(element.GDof);

% Gauss quadrature for bending part
[quadrature]=getQuadratureStructure;

% cycle for element
for e=1:element.numberElements
    % indice : nodal conectivities for each element
    indice=element.elementNodes(e,:);
    % indice : nodal conectivities for each element
    % elementDof: element degrees of freedom
    elementDof=[indice indice+element.numberNodes...
        indice+2*element.numberNodes];
    ndof=length(indice);

    % cycle for Gauss point
    for q=1:size(quadrature(2).weights,1)
        GaussPoint=quadrature(2).points
        xi=GaussPoint(1);
        eta=GaussPoint(2);

    % shape functions and derivatives
        [shapeFunction]=getShapeFunctionStructure(xi,eta)

    % Jacobian matrix, inverse of Jacobian,
    % derivatives w.r.t. x,y

```

```

[Jac]=JacobianStructure(element.nodeCoordinates(indice,:),...
    shapeFunction(1).naturalDerivatives)

% [B] matrix bending
B_b=zeros(3,3*ndof);
B_b(1,ndof+1:2*ndof) = Jac.derivatives(:,1)';
B_b(2,2*ndof+1:3*ndof)= Jac.derivatives(:,2)';
B_b(3,ndof+1:2*ndof) = Jac.derivatives(:,2)';
B_b(3,2*ndof+1:3*ndof)= Jac.derivatives(:,1)';

% stiffness matrix bending
K(elementDof,elementDof)=K(elementDof,elementDof)+ ...
    B_b'*element.C_bending*B_b*quadrature(2).weights(q)*...
    det(Jac.matrix);
end % Gauss point
end % element

% shear stiffness matrix

% cycle for element
for e=1:element.numberElements
    % indice : nodal conectivities for each element
    indice=element.elementNodes(e,:) ;
    % indice : nodal conectivities for each element
    % elementDof: element degrees of freedom
    elementDof=[indice indice+element.numberNodes...
        indice+2*element.numberNodes];
    ndof=length(indice);

    % cycle for Gauss point ! one Gauss point (reduced)
    for q=1:size(quadrature(1).weights,1)
        GaussPoint=quadrature(1).points
        xi=GaussPoint(1);
        eta=GaussPoint(2);

% shape functions and derivatives
        [shapeFunction]=getShapeFunctionStructure(xi,eta)

% Jacobian matrix, inverse of Jacobian,
% derivatives w.r.t. x,y

[Jac]=JacobianStructure(element.nodeCoordinates(indice,:),...
    shapeFunction(1).naturalDerivatives)

```

```

% [B] matrix shear
    B_s=zeros(2,3*ndof);
    B_s(1,1:ndof)      = Jac.derivatives(:,1)';
    B_s(2,1:ndof)      = Jac.derivatives(:,2)';
    B_s(1,ndof+1:2*ndof) = shapeFunction(1).shape;
    B_s(2,2*ndof+1:3*ndof)= shapeFunction(1).shape;

% stiffness matrix shear
    K(elementDof,elementDof)=K(elementDof,elementDof)+...
        B_s'*element.C_shear*B_s*quadrature(1).weights(q)*...
        det(Jac.matrix);
end    % gauss point
end    % element

end

%.....
function [force]=formForceVectorMindlinQ4Structure(element)

% computation of force vector
% for Mindlin plate element

% force : force vector
force=zeros(element.GDof,1);

% Gauss quadrature for bending part
[quadrature]=getQuadratureStructure;

% cycle for element
for e=1:element.numberElements
    % indice : nodal conectivities for each element
    indice=element.elementNodes(e,:)    ;

    % cycle for Gauss point
    for q=1:size(quadrature(2).weights,1)
        GaussPoint=quadrature(2).points
        xi=GaussPoint(1);
        eta=GaussPoint(2);

    % shape functions and derivatives
        [shapeFunction]=getShapeFunctionStructure(xi,eta)

```



```

% Jacobian matrix, inverse of Jacobian,
% derivatives w.r.t. x,y

    [Jac]=JacobianStructure(element.nodeCoordinates(indice,:),...
    shapeFunction(1).naturalDerivatives)

% force vector
    force(indice)=force(indice)+...
        shapeFunction(1).shape(q)*element.P...
        *det(Jac.matrix)*quadrature(2).weights(q);
end    % Gauss point
end    % element
end

% .....
function [quadrature]=getQuadratureStructure

% quadrature points and weights for Gauss quadrature
% quadrilaterals and triangles

% points: Gauss points
% weights: Gauss weights

% Structure quadrature
quadrature=struct()

% order = 1 (1 x 1)
quadrature(1).points=[0;0];
quadrature(1).weights=4;

% order = 2 (2 x 2)
quadrature(2).points=[...
    -0.577350269189626 -0.577350269189626;
    0.577350269189626 -0.577350269189626;
    0.577350269189626  0.577350269189626;
    -0.577350269189626  0.577350269189626];
quadrature(2).weights=[ 1;1;1;1];

% order = 3 (3 x 3)
quadrature(3).points=[...
    -0.774596669241483 -0.774596669241483;
    -0.774596669241483  0.0;

```



```

shapeFunction(1).naturalDerivatives=...
    1/4*[-(1-eta), -(1-xi);1-eta,    -(1+xi);
        1+eta,    1+xi;-(1+eta),    1-xi];

% T3 element
shapeFunction(2).shape=[1-xi-eta;xi;eta];
shapeFunction(2).naturalDerivatives=[-1,-1;1,0;0,1];

% Q9 element
shapeFunction(3).shape=1/4*[xi*eta*(xi-1)*(eta-1);
    xi*eta*(xi+1)*(eta-1);
    xi*eta*(xi+1)*(eta+1);
    xi*eta*(xi-1)*(eta+1);
    -2*eta*(xi+1)*(xi-1)*(eta-1);
    -2*xi*(xi+1)*(eta+1)*(eta-1);
    -2*eta*(xi+1)*(xi-1)*(eta+1);
    -2*xi*(xi-1)*(eta+1)*(eta-1);
    4*(xi+1)*(xi-1)*(eta+1)*(eta-1)];
shapeFunction(3).naturalDerivatives=...
    1/4*[eta*(2*xi-1)*(eta-1),xi*(xi-1)*(2*eta-1);
        eta*(2*xi+1)*(eta-1),xi*(xi+1)*(2*eta-1);
        eta*(2*xi+1)*(eta+1),xi*(xi+1)*(2*eta+1);
        eta*(2*xi-1)*(eta+1),xi*(xi-1)*(2*eta+1);
        -4*xi*eta*(eta-1),    -2*(xi+1)*(xi-1)*(2*eta-1);
        -2*(2*xi+1)*(eta+1)*(eta-1),-4*xi*eta*(xi+1);
        -4*xi*eta*(eta+1),    -2*(xi+1)*(xi-1)*(2*eta+1);
        -2*(2*xi-1)*(eta+1)*(eta-1),-4*xi*eta*(xi-1);
        8*xi*(eta^2-1),    8*eta*(xi^2-1)];

% Q8 element
shapeFunction(4).shape=[1/4*xi*(1-xi)*eta*(1-eta);
    -1/2*xi*(1-xi)*(1+eta)*(1-eta);
    -1/4*xi*(1-xi)*eta*(1+eta);
    1/2*(1+xi)*(1-xi)*(1+eta)*eta;
    1/4*xi*(1+xi)*eta*(1+eta);
    1/2*xi*(1+xi)*(1+eta)*(1-eta);
    -1/4*xi*(1+xi)*eta*(1-eta);
    -1/2*(1+xi)*(1-xi)*(1-eta)*eta];
shapeFunction(4).naturalDerivatives=...
    [1/4*eta*(-1+eta)*(-1+2*xi)  1/4*xi*(-1+xi)*(-1+2*eta);
    -1/2*(1+eta)*(-1+eta)*(-1+2*xi)  -xi*(-1+xi)*eta;
    1/4*eta*(1+eta)*(-1+2*xi)  1/4*xi*(-1+xi)*(1+2*eta);
    -xi*eta*(1+eta)  -1/2*(1+xi)*(-1+xi)*(1+2*eta);
    1/4*eta*(1+eta)*(1+2*xi)  1/4*xi*(1+xi)*(1+2*eta);

```

```

-1/2*(1+eta)*(-1+eta)*(1+2*xi) -xi*(1+xi)*eta;
1/4*eta*(-1+eta)*(1+2*xi) 1/4*xi*(1+xi)*(-1+2*eta);
-xi*eta*(-1+eta) -1/2*(1+xi)*(-1+xi)*(-1+2*eta)];

end % end function shapeFunctionQ4

% .....
function [Jac]=JacobianStructure(nodeCoordinates,natural
    Derivatives)

% Jac.matrix      : Jacobian matrix
% Jac.inv         : inverse of Jacobian Matrix
% Jac.derivatives : derivatives w.r.t. x and y
% naturalDerivatives : derivatives w.r.t. xi and eta
% nodeCoordinates : nodal coordinates at element level

Jac=struct();

Jac.matrix=nodeCoordinates'*naturalDerivatives;
Jac.inv=inv(Jac.matrix);
Jac.derivatives=naturalDerivatives*Jac.inv;

end % end function Jacobian

%.....

function prescribed=EssentialBCStructure(typeBC,element)

% essentialBoundary conditions for rectangular plates
xx=element.nodeCoordinates(:,1);
yy=element.nodeCoordinates(:,2);

switch typeBC
    case 'ssss'
fixedNodeW =find(xx==max(element.nodeCoordinates(:,1))|...
                xx==min(element.nodeCoordinates(:,1))|...
                yy==min(element.nodeCoordinates(:,2))|...
                yy==max(element.nodeCoordinates(:,2)));

fixedNodeTX =find(yy==max(element.nodeCoordinates(:,2))|...
                yy==min(element.nodeCoordinates(:,2)));
fixedNodeTY =find(xx==max(element.nodeCoordinates(:,1))|...

```

```

        xx==min(element.nodeCoordinates(:,1));
        case 'cccc'

fixedNodeW =find(xx==max(element.nodeCoordinates(:,1))|...
                xx==min(element.nodeCoordinates(:,1))|...
                yy==min(element.nodeCoordinates(:,2))|...
                yy==max(element.nodeCoordinates(:,2)));
fixedNodeTX =fixedNodeW;
fixedNodeTY =fixedNodeTX;
        case 'scsc'

fixedNodeW =find(xx==max(element.nodeCoordinates(:,1))|...
                xx==min(element.nodeCoordinates(:,1))|...
                yy==min(element.nodeCoordinates(:,2))|...
                yy==max(element.nodeCoordinates(:,2)));

fixedNodeTX =find(xx==max(element.nodeCoordinates(:,2))|...
                xx==min(element.nodeCoordinates(:,2)));
fixedNodeTY=[];
        case 'cccf'
fixedNodeW =find(xx==min(element.nodeCoordinates(:,1))|...
                yy==min(element.nodeCoordinates(:,2))|...
                yy==max(element.nodeCoordinates(:,2)));

fixedNodeTX =fixedNodeW;
fixedNodeTY =fixedNodeTX;
end

prescribed=[fixedNodeW;fixedNodeTX+element.numberNodes;...
            fixedNodeTY+2*element.numberNodes];
end

%.....

function displacements=solutionStructure(element)
% function to find solution in terms of global displacements

activeDof=setdiff([1:element.GDof]', ...
                [element.prescribed]));
U=element.stiffness(activeDof,activeDof)\...
    element.force(activeDof);
displacements=zeros(element.GDof,1);

```

```
displacements(activeDof)=U;
end
```

12.5 Free vibrations of Mindlin plates

By using the Hamilton principle [15], we may express the equations of motion of Mindlin plates as

$$\mathbf{M}\ddot{\mathbf{u}} + \mathbf{K}\mathbf{u} = \mathbf{f} \quad (12.28)$$

where $\mathbf{M}, \mathbf{K}, \mathbf{f}$ are the system mass and stiffness matrices, and the force vector, respectively, and $\ddot{\mathbf{u}}, \mathbf{u}$ are the accelerations and displacements. Assuming a harmonic motion we obtain the natural frequencies and the modes of vibration by solving the generalized eigenproblem [8]

$$(\mathbf{K} - \omega^2 \mathbf{M}) \mathbf{X} = \mathbf{0} \quad (12.29)$$

where ω is the natural frequency and $\mathbf{X}$ the mode of vibration.

By using the kinetic energy for the plate

$$T^e = \frac{1}{2} \int_A \rho \left[h \dot{w}^2 + \frac{h^3}{12} \dot{\theta}_x^2 + \frac{h^3}{12} \dot{\theta}_y^2 \right] dA \quad (12.30)$$

we may compute the mass matrix as [15]

$$\mathbf{M}^e = \int_A \rho \mathbf{N}^T \begin{bmatrix} h & 0 & 0 \\ 0 & \frac{h^3}{12} & 0 \\ 0 & 0 & \frac{h^3}{12} \end{bmatrix} \mathbf{N} dA \quad (12.31)$$

being the stiffness matrix already obtained before for static problems.

We consider a square plate (side a), with thickness-to-side ratio $h/a = 0.01$ and $h/a = 0.1$. The non-dimensional natural frequency is given by

$$\bar{\omega} = \omega_{mn} a \sqrt{\frac{\rho}{G}},$$

where ρ is the material density, G the shear modulus ($G = E/(2(1 + \nu))$), E the modulus of elasticity and ν the Poisson's coefficient. Indices m and n are the vibration half-waves in axes x and y . In this problem we consider fully simply-supported (SSSS) and fully clamped (CCCC) plates, as well as SCSC and CCCF plates where F means free side.

For CCCC and CCCF we use a shear correction factor $k = 0.8601$, while for SCSC plates we use $k = 0.822$. For SSSS plates we consider $k = 5/6$.

In table 12.2 we show the convergence of the fundamental frequency for CCCC plate with $h/a = 0.01, k = 0.8601, \nu = 0.3$. We obtain quite good agreement with the analytical solution [19].

In table 12.3 we show the convergence of the fundamental frequency for SSSS plate with $h/a = 0.01, k = 0.8333, \nu = 0.3$. Again, we obtain quite good agreement with a analytical solution [19]. Tables 12.4 and 12.5 consider $h/a = 1$ and in all of them results agree very well with analytical solution.

Tables 12.6 and 12.7 list the natural frequencies of a SSSS plate with $h/a = 0.1$ and $h/a = 0.01$, being $k = 0.833, \nu = 0.3$. Our finite element solution agrees with the tridimensional solution and analytical solution given by Mindlin [8].

Tables 12.8 and 12.9 compare natural frequencies with the Rayleigh-Ritz solution [8] and a solution by Liew [20].

Tables 12.10 and 12.11 compare natural frequencies for SCSC plate with $h/a = 0.1$ and $h/a = 0.01$, being $k = 0.822, \nu = 0.3$, respectively. Sides located at $x = 0; L$ are simply-supported.

Table 12.2 Convergence of natural frequency $\bar{\omega}$ for CCCC plate with $h/a = 0.01, k = 0.8601, \nu = 0.3$

10 × 10 Q4	0.1800	Analytical [19] 0.1754
15 × 15 Q4	0.1774	
20 × 20 Q4	0.1765	
25 × 25 Q4	0.1761	

Table 12.3 Convergence of natural frequency $\bar{\omega}$ for SSSS plate with $h/a = 0.01, k = 0.8333, \nu = 0.3$

10 × 10 Q4	0.0973	Analytical [19] 0.0963
15 × 15 Q4	0.0968	
20 × 20 Q4	0.0965	
25 × 25 Q4	0.0965	

Table 12.4 Convergence of natural frequency $\bar{\omega}$ for CCCC plate with $h/a = 0.1, k = 0.8601, \nu = 0.3$

10 × 10 Q4	1.6259	Analytical [19] 1.5940
15 × 15 Q4	1.6063	
20 × 20 Q4	1.5996	

Table 12.5 Convergence of natural frequency $\bar{\omega}$ for SSSS plate with $h/a = 0.1, k = 0.8333, \nu = 0.3$

10 × 10 Q4	0.9399	Analytical [19] 0.930
15 × 15 Q4	0.9346	
20 × 20 Q4	0.9327	

Table 12.6 Natural frequencies of a SSSS plate with $h/a = 0.1, k = 0.833, \nu = 0.3$

Mode no.	m	n	15×15 Q4	3D *	Mindlin *
1	1	1	0.9346	0.932	0.930
2	2	1	2.2545	2.226	2.219
3	1	2	2.2545	2.226	2.219
4	2	2	3.4592	3.421	3.406
5	3	1	4.3031	4.171	4.149
6	1	3	4.3031	4.171	4.149
7	3	2	5.3535	5.239	5.206
8	2	3	5.3535	5.239	5.206
9	4	1	6.9413	—	6.520
10	1	4	6.9413	—	6.520
11	3	3	7.0318	6.889	6.834
12	4	2	7.8261	7.511	7.446
13	2	4	7.8261	7.511	7.446

* analytical solution

Table 12.7 Natural frequencies of a SSSS plate with $h/a = 0.01, k = 0.833, \nu = 0.3$

Mode no.	m	n	20×20 Q4	Mindlin *
1	1	1	0.0965	0.0963
2	2	1	0.2430	0.2406
3	1	2	0.2430	0.2406
4	2	2	0.3890	0.3847
5	3	1	0.4928	0.4807
6	1	3	0.4928	0.4807
7	3	2	0.6380	0.6246
8	2	3	0.6380	0.6246
9	4	1	0.8550	0.8156
10	1	4	0.8550	0.8156
11	3	3	0.8857	0.8640
12	4	2	0.9991	0.9592
13	2	4	0.9991	0.9592

(* – analytical solution)

Tables 12.12 and 12.13 compare natural frequencies for CCCF plates with $h/a = 0.1$ and $h/a = 0.01$, respectively, being $k = 0.822, \nu = 0.3$. Side located at $x = L$ is free.

The present finite element results are quite accurate.

Figure 12.3 shows the modes of vibration for a CCCC plate with $h/a = 0.1$, using 10×10 Q4 elements.

Figure 12.4 shows the modes of vibration for a SSSS plate with $h/a = 0.1$, using 10×10 Q4 elements.

Figure 12.5 shows the modes of vibration for a SCSC plate with $h/a = 0.01$, using 15×15 Q4 elements.

Table 12.8 Natural frequencies of a CCCC plate with $h/a = 0.1, k = 0.8601, \nu = 0.3$

Mode no.	m	n	20×20 Q4	Rayleigh-Ritz [19]	Liew et al. [20]
1	1	1	1.5955	1.5940	1.5582
2	2	1	3.0662	3.0390	3.0182
3	1	2	3.0662	3.0390	3.0182
4	2	2	4.2924	4.2650	4.1711
5	3	1	5.1232	5.0350	5.1218
6	1	3	5.1730	5.0780	5.1594
7	3	2	6.1587		6.0178
8	2	3	6.1587		6.0178
9	4	1	7.6554		7.5169
10	1	4	7.6554		7.5169
11	3	3	7.7703		7.7288
12	4	2	8.4555		8.3985
13	2	4	8.5378		8.3985

Table 12.9 Natural frequencies of a CCCC plate with $h/a = 0.01, k = 0.8601, \nu = 0.3$

Mode no.	m	n	20×20 Q4	Rayleigh-Ritz [19]	Liew et al. [20]
1	1	1	0.175	0.1754	0.1743
2	2	1	0.3635	0.3576	0.3576
3	1	2	0.3635	0.3576	0.3576
4	2	2	0.5358	0.5274	0.5240
5	3	1	0.6634	0.6402	0.6465
6	1	3	0.6665	0.6432	0.6505
7	3	2	0.8266		0.8015
8	2	3	0.8266		0.8015
9	4	1	1.0875		1.0426
10	1	4	1.0875		1.0426
11	3	3	1.1049		1.0628
12	4	2	1.2392		1.1823
13	2	4	1.2446		1.1823

Table 12.10 Natural frequencies for SCSC plate with $h/a = 0.1, k = 0.822, \nu = 0.3$

Mode no.	m	n	15×15 Q4	Mindlin solution [8]
1	1	1	1.2940	1.302
2	2	1	2.3971	2.398
3	1	2	2.9290	2.888
4	2	2	3.8394	3.852
5	3	1	4.3475	4.237
6	1	3	5.1354	4.936
7	3	2	5.5094	
8	2	3	5.8974	
9	4	1	6.9384	
10	1	4	7.2939	
11	3	3	7.7968	
12	4	2	7.8516	
13	2	4	8.4308	

Table 12.11 Natural frequencies for SCSC plate with $h/a = 0.01, k = 0.822, \nu = 0.3$

Mode no.	m	n	15×15 Q4	Mindlin solution [8]
1	1	1	0.1424	0.1411
2	2	1	0.2710	0.2668
3	1	2	0.3484	0.3377
4	2	2	0.4722	0.4608
5	3	1	0.5191	0.4979
6	1	3	0.6710	0.6279
7	3	2	0.7080	
8	2	3	0.7944	
9	4	1	0.8988	
10	1	4	1.0228	
11	3	3	1.0758	
12	4	2	1.1339	
13	2	4	1.2570	

Table 12.12 Natural frequencies for CCCF plate with $h/a = 0.1, k = 0.8601, \nu = 0.3$

Mode no.	m	n	15×15 Q4	Mindlin solution [8]
1	1	1	1.0923	1.089
2	2	1	1.7566	1.758
3	1	2	2.7337	2.673
4	2	2	3.2591	3.216
5	3	1	3.3541	3.318
6	1	3	4.6395	4.615
7	3	2	4.9746	
8	2	3	5.4620	
9	4	1	5.5245	
10	1	4	6.5865	
11	3	3	6.6347	
12	4	2	7.6904	
13	2	4	8.1626	

Table 12.13 Natural frequencies for CCCF plate with $h/a = 0.01, k = 0.8601, \nu = 0.3$

Mode no.	m	n	15×15 Q4	Mindlin solution [8]
1	1	1	0.1180	0.1171
2	2	1	0.1967	0.1951
3	1	2	0.3193	0.3093
4	2	2	0.3830	0.3740
5	3	1	0.4031	0.3931
6	1	3	0.5839	0.5695
7	3	2	0.6387	
8	2	3	0.7243	
9	4	1	0.8817	
10	1	4	0.9046	
11	3	3	1.0994	
12	4	2	1.1407	
13	2	4	1.1853	

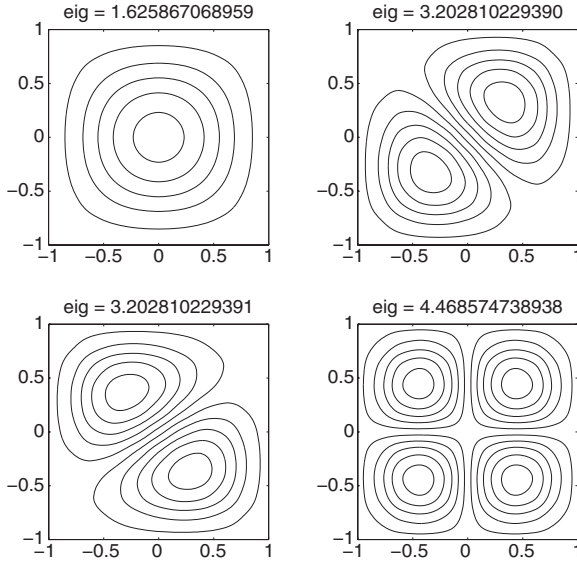


Fig. 12.3 Modes of vibration for a CCCC plate with $h/a = 0.1$, using 10×10 Q4 elements

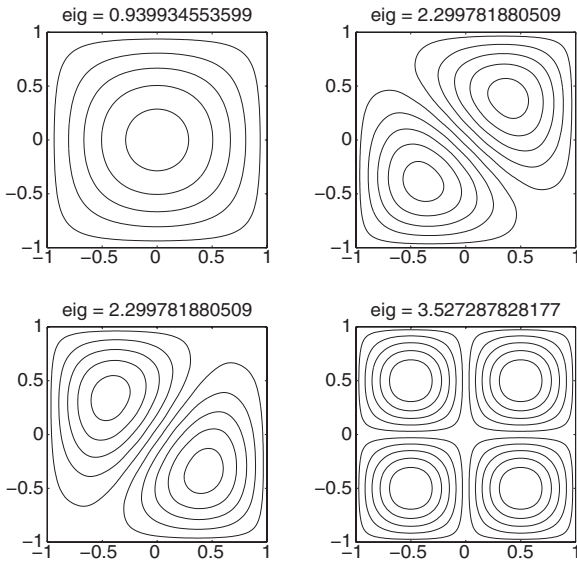


Fig. 12.4 Modes of vibration for a SSSS plate with $h/a = 0.1$, using 10×10 Q4 elements

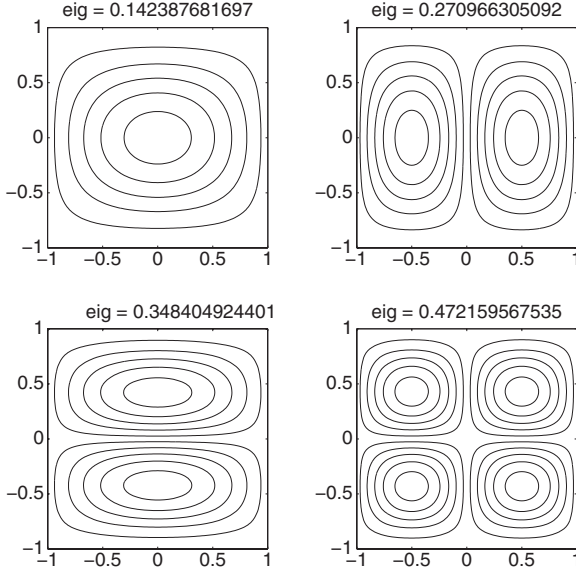


Fig. 12.5 Modes of vibration for a SCSC plate with $h/a = 0.01$, using 15×15 Q4 elements

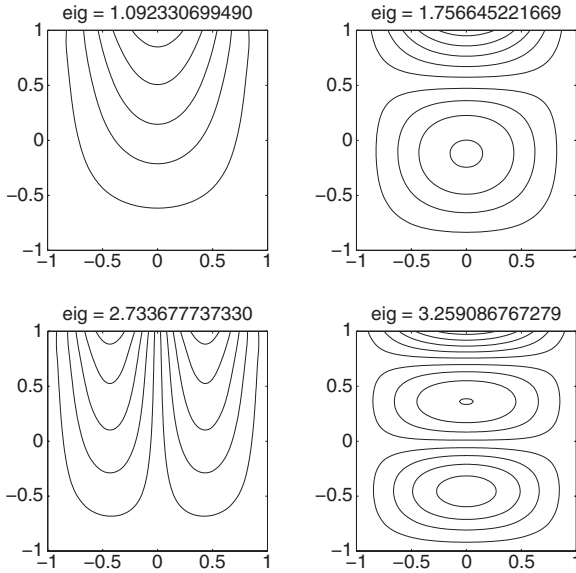


Fig. 12.6 Modes of vibration for a CCCF plate with $h/a = 0.1$, using 15×15 Q4 elements

Figure 12.6 shows the modes of vibration for a CCCF plate with $h/a = 0.1$, using 15×15 Q4 elements.

The MATLAB code (`problem19Vibrations.m`), solves the free vibration problem of Mindlin plates. The user is requested to change input details according to the problem.

```
%.....

% MATLAB codes for Finite Element Analysis
% problem19vibrations.m
% Mindlin plate in free vibrations
% antonio ferreira 2008

% clear memory
clear all;colordef white;clf

% materials
E = 10920;      poisson = 0.30;
thickness=0.1;
I=thickness^3/12;
rho=1;
kapa=0.8601; % cccc / cccf case
%kapa=0.822; % scsc case
kapa=5/6; % ssss case

% matrix C
% bending part
C_bending=I*E/(1-poisson^2)*...
    [1 poisson 0;poisson 1 0;0 0 (1-poisson)/2];
% shear part
C_shear=kapa*thickness*E/2/(1+poisson)*eye(2);

% load
P = -1;

%Mesh generation
L = 1;
numberElementsX=10;
numberElementsY=10;
numberElements=numberElementsX*numberElementsY;

[nodeCoordinates, elementNodes] = ...
    rectangularMesh(L,L,numberElementsX,numberElementsY);
xx=nodeCoordinates(:,1);
yy=nodeCoordinates(:,2);
```

```

drawingMesh(nodeCoordinates,elementNodes,'Q4','k-');
axis off
numberNodes=size(xx,1);

% GDof: global number of degrees of freedom
GDof=3*numberNodes;

% computation of the system stiffness and mass matrices
[stiffness]=...
    formStiffnessMatrixMindlinQ4(GDof,numberElements,...
    elementNodes,numberNodes,nodeCoordinates,C_shear,...
    C_bending,thickness,I);

[mass]=...
formMassMatrixMindlinQ4(GDof,numberElements,...
elementNodes,numberNodes,nodeCoordinates,thickness,rho,I);

% % boundary conditions
[prescribedDof,activeDof,fixedNodeW]=...
EssentialBC('cccc',GDof,xx,yy,nodeCoordinates,numberNodes)

G=E/2.6;
% V : mode shape
% D : frequency
%
numberOfModes=12;
[V,D] = eig(stiffness(activeDof,activeDof),...
    mass(activeDof,activeDof));
D = diag(sqrt(D)*L*sqrt(rho/G));
[D,ii] = sort(D); ii = ii(1:numberOfModes);
VV = V(:,ii);
activeDofW=setdiff([1:numberNodes]',[fixedNodeW]);
NNN=size(activeDofW);

    VVV(1:numberNodes,1:12)=0;
    for i=1:12
        VVV(activeDofW,i)=VV(1:NNN,i);
    end

NN=numberNodes;N=sqrt(NN);
x=linspace(-L,L,numberElementsX+1);
y=linspace(-L,L,numberElementsY+1);

```

```
% drawing Eigenmodes
drawEigenmodes2D(x,y,VVV,NN,N,D)
```

This code calls function `formMassMatrixMindlinQ4.m` which computes the mass matrices of the Mindlin Q4 element. The code for computing the stiffness matrix has already been presented.

```
%.....

function [mass]=...
    formMassMatrixMindlinQ4(GDof,numberElements,...
        elementNodes,numberNodes,nodeCoordinates,thickness,rho,I)

% computation of mass matrix
% for Mindlin plate element

% mass : mass matrix
mass=zeros(GDof);

% Gauss quadrature for bending part
[gaussWeights,gaussLocations]=gaussQuadrature('complete');

% cycle for element
for e=1:numberElements
    % indice : nodal condofectivities for each element
    indice=elementNodes(e,:);
    ndof=length(indice);

    % cycle for Gauss point
    for q=1:size(gaussWeights,1)
        GaussPoint=gaussLocations(q,:);
        xi=GaussPoint(1);
        eta=GaussPoint(2);

        % shape functions and derivatives
        [shapeFunction,naturalDerivatives]=shapeFunctionQ4(xi,eta)

        % Jacobian matrix, inverse of Jacobian,
```

```

% derivatives w.r.t. x,y
[Jacob,invJacobian,XYderivatives]=...
    Jacobian(nodeCoordinates(indice,:),naturalDerivatives);

% [B] matrix bending
B_b=zeros(3,3*ndof);
B_b(1,ndof+1:2*ndof) = XYderivatives(:,1)';
B_b(2,2*ndof+1:3*ndof)= XYderivatives(:,2)';
B_b(3,ndof+1:2*ndof) = XYderivatives(:,2)';
B_b(3,2*ndof+1:3*ndof)= XYderivatives(:,1)';

% mass matrix

mass(indice,indice)=mass(indice,indice)+...
    shapeFunction*shapeFunction'*thickness*...
    rho*gaussWeights(q)*det(Jacob);
mass(indice+numberNodes,indice+numberNodes)=...
    mass(indice+numberNodes,indice+numberNodes)+...
    shapeFunction*shapeFunction'*I*...
    rho*gaussWeights(q)*det(Jacob);
mass(indice+2*numberNodes,indice+2*numberNodes)=...
    mass(indice+2*numberNodes,indice+2*numberNodes)+...
    shapeFunction*shapeFunction'*I*...
    rho*gaussWeights(q)*det(Jacob);

end % Gauss point
end % element

```

12.6 Buckling analysis of Mindlin plates

In this section we formulate and implement the buckling analysis of Mindlin plates. After presenting the basic finite element formulation, we present a MATLAB code for buckling analysis of a simply-supported isotropic square plate under uniaxial initial stress.

The strain energy for an initially stressed Mindlin plate, after neglecting terms with third and higher powers in displacement gradients, can be written as [8]

$$U = \frac{1}{2} \int_V \epsilon^T \mathbf{Q} \epsilon dV + \frac{1}{2} \int_V \gamma^T \mathbf{Q}_c \gamma dV + \int_V (\sigma^0)^T \epsilon^L dV \quad (12.32)$$

where

$$\boldsymbol{\epsilon}^L = \begin{bmatrix} \frac{1}{2} \left(\left(\frac{\partial u}{\partial x} \right)^2 + \left(\frac{\partial v}{\partial x} \right)^2 + \left(\frac{\partial w}{\partial x} \right)^2 \right) \\ \frac{1}{2} \left(\left(\frac{\partial u}{\partial y} \right)^2 + \left(\frac{\partial v}{\partial y} \right)^2 + \left(\frac{\partial w}{\partial y} \right)^2 \right) \\ \frac{\partial u}{\partial x} \frac{\partial u}{\partial y} + \frac{\partial v}{\partial x} \frac{\partial v}{\partial y} + \frac{\partial w}{\partial x} \frac{\partial w}{\partial y} \end{bmatrix} \quad (12.33)$$

Integrating over the plate thickness we obtain U as

$$\begin{aligned} U = & \frac{1}{2} \int_A \boldsymbol{\epsilon}^T \mathbf{D}_f \boldsymbol{\epsilon} dA + \frac{1}{2} \int_A \boldsymbol{\gamma}^T \mathbf{D}_c \boldsymbol{\gamma} dA + \frac{1}{2} \int_A \left[\left(\frac{\partial w}{\partial x} \right) \left(\frac{\partial w}{\partial y} \right) \right] \hat{\boldsymbol{\sigma}}^{0T} \begin{bmatrix} \left(\frac{\partial w}{\partial x} \right) \\ \left(\frac{\partial w}{\partial y} \right) \end{bmatrix} h dA \\ & + \frac{1}{2} \int_A \left[\left(\frac{\partial \theta_x}{\partial x} \right) \left(\frac{\partial \theta_x}{\partial y} \right) \right] \hat{\boldsymbol{\sigma}}^{0T} \begin{bmatrix} \left(\frac{\partial \theta_x}{\partial x} \right) \\ \left(\frac{\partial \theta_x}{\partial y} \right) \end{bmatrix} \frac{h^3}{12} dA \\ & + \frac{1}{2} \int_A \left[\left(\frac{\partial \theta_y}{\partial x} \right) \left(\frac{\partial \theta_y}{\partial y} \right) \right] \hat{\boldsymbol{\sigma}}^{0T} \begin{bmatrix} \left(\frac{\partial \theta_y}{\partial x} \right) \\ \left(\frac{\partial \theta_y}{\partial y} \right) \end{bmatrix} \frac{h^3}{12} dA \quad (12.34) \end{aligned}$$

where

$$\hat{\boldsymbol{\sigma}}^0 = \begin{bmatrix} \sigma_x^0 & \tau_{xy}^0 \\ \tau_{xy}^0 & \sigma_y^0 \end{bmatrix} \quad (12.35)$$

The stability problem involves the solution of the eigenproblem

$$[\mathbf{K} - \lambda \mathbf{K}_G] \mathbf{a}^i = 0, \quad i = 1, 2, \dots, r \quad (12.36)$$

where $\mathbf{K}$ is the global stiffness matrix, $\mathbf{K}_G$ is the geometric matrix and λ is a constant by which the in-plane loads must be multiplied to cause buckling. Vector $\mathbf{a}^i$ is the i -th buckling mode and r is the total number of degrees of freedom. Thus the buckling loads can be found by solving the eigenproblem in (12.36).

The geometric stiffness matrix $\mathbf{K}_G$ may be written as [8].

$$K_G = K_{Gb} + K_{Gs} \quad (12.37)$$

where the bending contribution is given by

$$K_{Gb} = \int_{-1}^1 \int_{-1}^1 \mathbf{G}_b^T \hat{\boldsymbol{\sigma}}^0 \mathbf{G}_b h \det \mathbf{J} d\xi d\eta \quad (12.38)$$

and for a given node i

$$\mathbf{G}_b = \begin{bmatrix} \frac{\partial N}{\partial x} & 0 & 0 \\ \frac{\partial N}{\partial y} & 0 & 0 \end{bmatrix} \quad (12.39)$$

The shear contribution is given as

$$K_{Gs} = \int_{-1}^1 \int_{-1}^1 \mathbf{G}_{s1}^T \hat{\boldsymbol{\sigma}}^0 \mathbf{G}_{s1} \frac{h^3}{12} \det \mathbf{J} d\xi d\eta + \int_{-1}^1 \int_{-1}^1 \mathbf{G}_{s2}^T \hat{\boldsymbol{\sigma}}^0 \mathbf{G}_{s2} \frac{h^3}{12} \det \mathbf{J} d\xi d\eta \quad (12.40)$$

where for a given node i

$$\mathbf{G}_{s1} = \begin{bmatrix} 0 & \frac{\partial N}{\partial x} & 0 \\ 0 & \frac{\partial N}{\partial y} & 0 \end{bmatrix} \quad (12.41)$$

$$\mathbf{G}_{s2} = \begin{bmatrix} 0 & 0 & \frac{\partial N}{\partial x} \\ 0 & 0 & \frac{\partial N}{\partial y} \end{bmatrix} \quad (12.42)$$

Although the shear contribution for the geometric stiffness matrix is negligible for thin plates, its effects can be significant for thicker plates.

Table 12.14 summarizes results for simply supported square plates of various thicknesses under uniaxial σ_{xx} initial stress. We consider a 10×10 Q4 mesh (figure 12.7), and compare present the finite element formulation with closed form solution [8]. The schematic geometry, loads and boundary conditions are illustrated in figure 12.8.

In figures 12.9–12.11 the eigenmodes are illustrated, for $a/h = 10$.

The MATLAB code `problem19Buckling.m` computes the problem of a Mindlin plate under compressive, uniaxial load. Note that the code is given as a function in order to be able to incorporate other functions in the same file.

Table 12.14 Buckling factors $k_b = a^2 F_{cr} / \pi^2 D_{f11}$ for a simply supported square plate under uniaxial initial stress ($\nu = 0.3$)

a/h	Closed form solution	Present FE formulation
1,000	4.000	4.0666
20	3.944	3.9930
10	3.786	3.7890
5	3.264	3.1665

Fig. 12.7 Buckling of Mindlin plate: 10×10 Q4 mesh

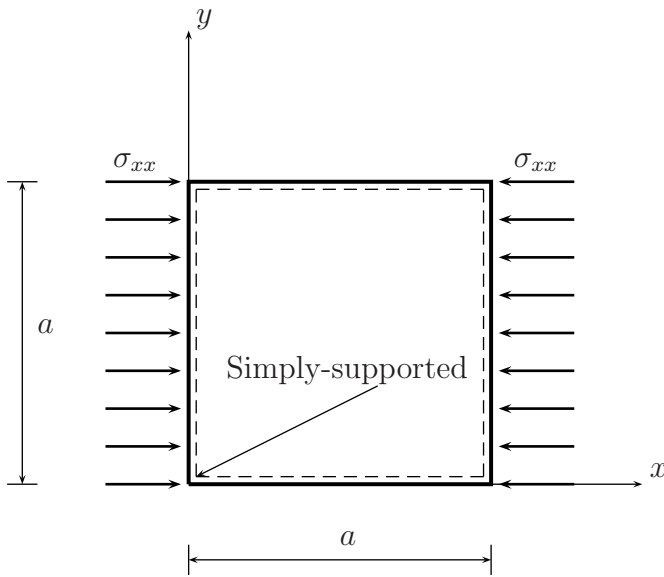
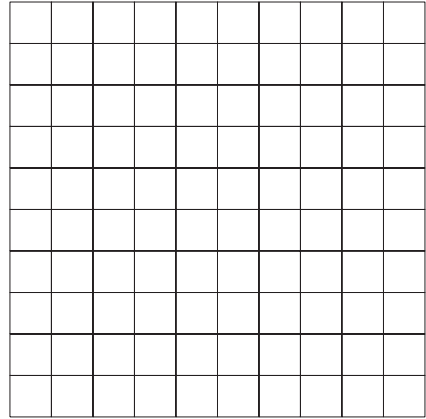


Fig. 12.8 Buckling problem: a Mindlin plate under uniaxial initial stress

This code calls function `formGeometricStiffnessMindlinQ4.m` for the computation of the geometric stiffness matrix.

```
%.....  
  
% MATLAB codes for Finite Element Analysis
```

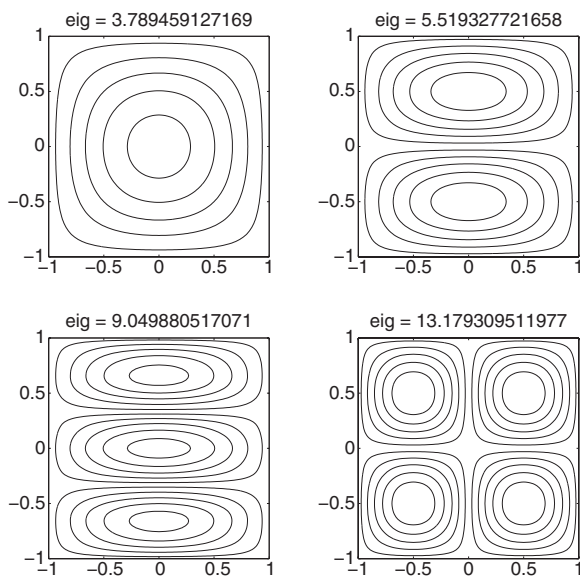


Fig. 12.9 Buckling modes (1 to 4)

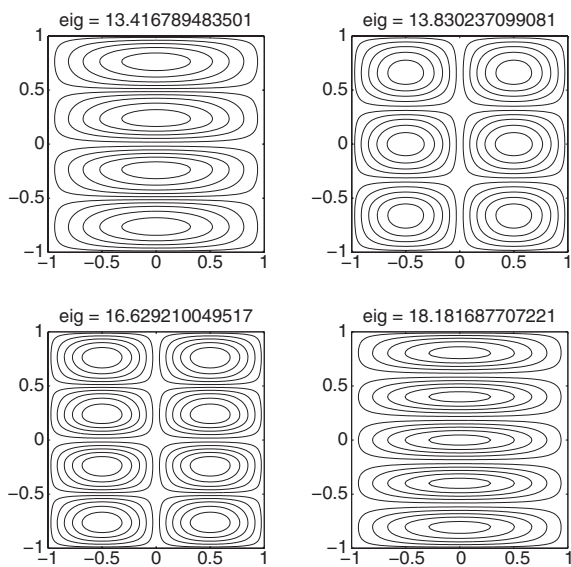


Fig. 12.10 Buckling modes (5 to 8)

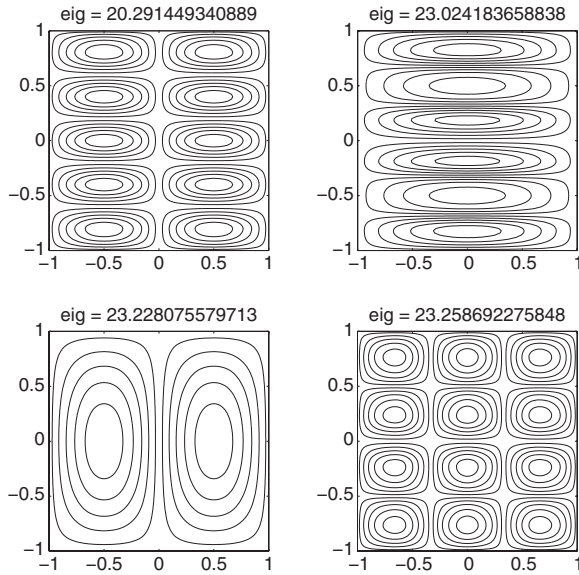


Fig. 12.11 Buckling modes (9 to 12)

```
% problem19Buckling.m
% this function performs buckling analysis
% of Mindlin plates using 3 degrees of freedom per node
% antonio ferreira 2008

clear all
colordef white

% material properties
% modulusOfElasticity = Young's modulus
% PoissonRatio       = Poisson's ratio

modulusOfElasticity = 10920; % Young
PoissonRatio = 0.30; % coef. Poisson

% L: side lenght
L = 1;

thickness=0.001;
I=thickness^3/12;

% kapa: shear correction factor
```

```

kapa=5/6;

% constitutive matrix
% bending part
C_bending=...
    I*modulusOfElasticity/(1-PoissonRatio^2)*...
    [    1                PoissonRatio                0 ;
      PoissonRatio          1                0 ;
         0                0      (1-PoissonRatio)/2 ];

% shear part
C_shear=...
kapa*thickness*modulusOfElasticity/2/(1+PoissonRatio)*eye(2);

% initial stress matrix
sigmaX=1/thickness;
sigmaXY=0;
sigmaY=0;
sigmaMatrix=[ sigmaX sigmaXY; sigmaXY sigmaY];

% mesh generation ...
% numberElementsX: number of elements in x
% numberElementsY: number of elements in y
numberElementsX=10;
numberElementsY=10;
% number of elements
numberElements=numberElementsX*numberElementsY;
[nodeCoordinates, elementNodes] = ...
    rectangularMesh(L, L, numberElementsX, numberElementsY);
xx=nodeCoordinates(:,1);    yy=nodeCoordinates(:,2);
figure
drawingMesh(nodeCoordinates,elementNodes,'Q4','k-');
axis off

numberNodes=size(xx,1);    % number of nodes
GDof=3*numberNodes;        % total number of DOFs

% stiffness and geometric stiffness matrices
[stiffness]=...
    formStiffnessMatrixMindlinQ4(GDof,numberElements,...
    elementNodes,numberNodes,nodeCoordinates,C_shear,...
    C_bending,thickness,I);

```

```

[geometric]=...
formGeometricStiffnessMindlinQ4(GDof,numberElements,...
elementNodes,numberNodes,nodeCoordinates,sigmaMatrix,thickness);

% Essential boundary conditions
[prescribedDof,activeDof,fixedNodeW]=...
EssentialBC('ssss',GDof,xx,yy,nodeCoordinates,numberNodes);

% buckling analysis ...

% perform eigenproblem
[V1,D1] = eig(stiffness(activeDof,activeDof),...
    geometric(activeDof,activeDof));
D1 = diag(D1);
% drawing eigenmodes
numberOfModes=12;
% sort out eigenvalues
[D1,ii] = sort(D1); ii = ii(1:numberOfModes);
VV = V1(:,ii);
activeDofW=setdiff([1:numberNodes]',[fixedNodeW]);
NNN=size(activeDofW);

% normalize results
disp('D1(1)/pi/pi/C_bending(1,1)')
D1(1)/pi/pi/C_bending(1,1)
D1(1)*pi*pi*C_bending(1,1)

VVV(1:numberNodes,1:numberOfModes)=0;
for i=1:numberOfModes
    VVV(activeDofW,i)=VV(1:NNN,i);
end
%
NN=numberNodes;N=sqrt(NN);
x=linspace(-L,L,numberElementsX+1);
y=linspace(-L,L,numberElementsY+1);

D1=D1/pi/pi/C_bending(1,1);
% drawing Eigenmodes
drawEigenmodes2D(x,y,VVV,NN,N,D1)

```

```

%.....

function [KG]=...
    formGeometricStiffnessMindlinQ4(GDof,numberElements,...
        elementNodes,numberNodes,nodeCoordinates,sigmaMatrix,thickness)

% computation of geometric stiffness
% for Mindlin plate element

% KG : geometric matrix
KG=zeros(GDof);

% Gauss quadrature for bending part
[gaussWeights,gaussLocations]=gaussQuadrature('reduced');

% cycle for element
for e=1:numberElements
    % indice : nodal condofectivities for each element
    % elementDof: element degrees of freedom
    indice=elementNodes(e,:);
    elementDof=[indice indice+numberNodes indice+2*numberNodes];
    ndof=length(indice);

    % cycle for Gauss point
    for q=1:size(gaussWeights,1)
        GaussPoint=gaussLocations(q,:);
        xi=GaussPoint(1);
        eta=GaussPoint(2);

% shape functions and derivatives
        [shapeFunction,naturalDerivatives]=shapeFunctionQ4(xi,eta)

% Jacobian matrix, inverse of Jacobian,
% derivatives w.r.t. x,y
        [Jacob,invJacobian,XYderivatives]=...
            Jacobian(nodeCoordinates(indice,:),naturalDerivatives);

% geometric matrix
        G_b=zeros(2,3*ndof);
        G_b(1,1:ndof) = XYderivatives(:,1)';
        G_b(2,1:ndof) = XYderivatives(:,2)';
        KG(elementDof,elementDof)=KG(elementDof,elementDof)+ ...

```



```

        G_b'*sigmaMatrix*thickness*G_b*gaussWeights(q)*det(Jacob);

    end % Gauss point

end % element

% shear stiffness matrix

% Gauss quadrature for shear part
[W,Q]=gaussQuadrature('reduced');

% cycle for element
for q=1:size(gaussWeights,1)
    GaussPoint=gaussLocations(q,:);
    xi=GaussPoint(1);
    eta=GaussPoint(2);

% shape functions and derivatives
    [shapeFunction,naturalDerivatives]=shapeFunctionQ4(xi,eta)

% Jacobian matrix, inverse of Jacobian,
% derivatives w.r.t. x,y
    [Jacob,invJacobian,XYderivatives]=...
        Jacobian(nodeCoordinates(indice,:),naturalDerivatives);

% Geometric matrix
    G_s1=zeros(2,3*ndof);
    G_s1(1,ndof+1:2*ndof) = XYderivatives(:,1)';
    G_s1(2,ndof+1:2*ndof) = XYderivatives(:,2)';
    KG(elementDof,elementDof) =KG(elementDof,elementDof)+ ...
        G_s1'*sigmaMatrix*thickness^3/12*G_s1*...
        gaussWeights(q)*det(Jacob);

    G_s2=zeros(2,3*ndof);
    G_s2(1,2*ndof+1:3*ndof) = XYderivatives(:,1)';
    G_s2(2,2*ndof+1:3*ndof) = XYderivatives(:,2)';
    KG(elementDof,elementDof) =KG(elementDof,elementDof)+ ...
        G_s2'*sigmaMatrix*thickness^3/12*G_s2*...
        gaussWeights(q)*det(Jacob);

    end % gauss point
end % element

```

Codes call function drawEigenmodes2D.p. Note that p-codes are not editable.